FOCP II

STUDENT EXCUSE GENERATOR

ANIMESH PASRIJA 25CSU028

PROMPT AND OUTPUT 1:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Write a simple C++ program that generates a student excuse based on a name.
Requirements:

Ask the user to enter their name
Store at least 5 excuse messages
Display one excuse message

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Display one excuse message

Here is a beginner-friendly C++ program that generates a random excuse. I've applied the same logic we discussed for the roast generator—using `getline` for full names and `srand` for variety—to ensure the program is robust and professional.

The Logic

1. **Seeding:** We use the system time to ensure the "random" choice changes every time you run the code.
2. **Input:** We use `std::getline` to make sure we don't cut off the student's last name.
3. **Storage:** We use a `std::vector` to hold the strings.
4. **Selection:** We use the modulo operator (`%`) to pick a valid index from our list of excuses.

The Student Excuse Generator

C++

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

C++

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <ctime>
#include <cstdlib>

int main() {
    // Seed the random number generator
    std::srand(static_cast<unsigned int>(std::time(0)));

    std::string studentName;

    // Get the student's name
    std::cout << "--- Official Student Excuse Generator ---" << std::endl;
    std::cout << "Enter student name: ";
    std::getline(std::cin, studentName);

    // Simple validation
    if (studentName.empty()) {
        studentName = "The Student";
    }

    // List of excuses
    std::vector<std::string> excuses = {
        " was unable to attend because their dog actually ate their laptop.",
        " is absent because they got stuck in a recursive loop while coding.",
        " cannot submit the work because their printer became sentient and refus
        " is running late because they accidentally joined a parade on the way t
        " missed the deadline because a solar flare wiped their hard drive (and
    };

    // Randomly pick one excuse
    int index = std::rand() % excuses.size();

    // Display the result
    std::cout << "\nTo whom it may concern," << std::endl;
    std::cout << studentName << excuses[index] << std::endl;
    std::cout << "\nPlease excuse this unusual circumstance." << std::endl;
```

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

Key Takeaways

- **Safety:** I added a small check so that if the user hits enter without a name, it defaults to "The Student" instead of leaving a blank space.
- **Readability:** The excuses start with a space (e.g., `" was..."`) so that when you print `studentName` followed by the excuse, the sentence is formatted correctly.
- **Extensibility:** You can easily add a 6th or 10th excuse to the `vector` without changing any other line of code, thanks to `excuses.size()`.

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

PROMPT AND OUTPUT 2:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Improve the previous program:

Store at least 10 excuse templates in a vector
Each template should contain a placeholder {name}
Replace {name} with the user input
Randomly select one excuse using rand()
Explain the logic before writing code.

To improve this program, we will apply the "Template and Replace" pattern. This is a common technique in software development where you create a generic string and inject specific data into it at runtime.

The Logic

1. **Template Storage**: Instead of just appending a name to the start of a sentence, we store the excuses as complete sentences containing the tag `{name}`. This allows the name to appear anywhere in the excuse (beginning, middle, or end), making the messages feel more natural.
2. **Randomization Logic**: We use `std::srand(time(0))` to ensure the "random" sequence is different every time the program runs. We then use the modulo operator (`%`) with `vec.size()` to pick a random index within the bounds of our 10-item list.
3. **The Find-and-Replace Algorithm**:
   - We use `selectedExcuse.find("{name}")` to get the exact memory address (index) where the placeholder starts.
   - We use `selectedExcuse.replace(...)` to "cut" the placeholder out and "paste" the user's name in its place.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

C++

```cpp
int main() {
    // Seed the random number generator
    std::srand(static_cast<unsigned int>(std::time(0)));

    std::string studentName;
    std::cout << "--- Professional Excuse Generator ---\n";
    std::cout << "Enter the student's name: ";
    std::getline(std::cin, studentName);

    // Input validation for empty names
    if (studentName.empty()) {
        studentName = "The student";
    }

    // 10 Excuse templates with {name} placeholder
    std::vector<std::string> excuses = {
        "Please excuse {name}, as their alarm clock was hijacked by a mischievou
        "Due to a sudden localized gravity failure, {name} was unable to leave t
        "The textbook required by {name} was unfortunately seized by pirates.",
        "I am writing to inform you that {name} is late because they had to solv
        "A rogue cloud followed {name} all morning, making travel impossible.",
        "The internet connection at {name}'s house was tired and needed a nap.",
        "Please forgive {name}; they accidentally entered a witness protection p
        "A group of ducks blocked the driveway, and {name} didn't want to be rud
        "While studying, {name} accidentally discovered a portal to the year 192
        "The dog didn't eat the homework, but he did hide {name}'s shoes in the
    };

    // Pick a random template
    int index = std::rand() % excuses.size();
    std::string selectedExcuse = excuses[index];

    // Placeholder replacement logic
    std::string target = "{name}";
    size_t pos = selectedExcuse.find(target);

    if (pos != std::string::npos) {
        selectedExcuse.replace(pos, target.length(), studentName);
    }

    // Final Output
    std::cout << "\nGenerated Note:\n\"" << selectedExcuse << "\"\n" << std::endl
```

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

Why this is a "Clean" Solution:

- **Decoupling**: The logic of *picking* the excuse is separate from the logic of *customizing* it.
- **Scalability**: You could add 100 more excuses to the vector, and the replacement logic would still work perfectly without modification.
- **Memory Safety**: By using `std::string::npos` (which basically means "not found"), we ensure that if a template accidentally leaves out the `{name}` tag, the program won't crash trying to replace it.

Does this template system make sense, or would you like to see how to add multiple different placeholders like `{date}` or `{subject}`?

PROMPT AND OUTPUT 3:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

The program is not reading full names correctly.
Fix the issue by:

Using getline instead of cin
Handling input properly
Explain the mistake and provide corrected code.

The issue you're encountering is a fundamental part of how C++ handles input streams. When you use `std::cin >> variable`, it uses **whitespace** (spaces, tabs, or newlines) as a delimiter.

The Mistake: `std::cin >>`

If you type "Sarah Connor," `std::cin` reads "Sarah," sees the space, and stops. "Connor" stays behind in the input buffer, often causing the program to skip future prompts or display incomplete data.

The Fix: `std::getline`

The `std::getline()` function is designed to read an entire line of text until it encounters a newline character (`\n`), which is created when you press the Enter key. This allows the string to include spaces.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

C++

```cpp
#include <cstdlib>

int main() {
    std::srand(static_cast<unsigned int>(std::time(0)));

    std::string studentName;
    std::cout << "--- Professional Excuse Generator ---\n";

    // PROBLEM FIX: Use getline(stream, variable) to capture the full line
    std::cout << "Enter the student's full name: ";
    std::getline(std::cin, studentName);

    // Handle empty input to prevent blank excuses
    if (studentName.empty()) {
        studentName = "The student";
    }

    std::vector<std::string> excuses = {
        "Please excuse {name}, as their alarm clock was hijacked by a mischievou
        "Due to a sudden localized gravity failure, {name} was unable to leave t
        "The textbook required by {name} was unfortunately seized by pirates.",
        "I am writing to inform you that {name} is late because they had to solv
        "A rogue cloud followed {name} all morning, making travel impossible.",
        "The internet connection at {name}'s house was tired and needed a nap.",
        "Please forgive {name}; they accidentally entered a witness protection p
        "A group of ducks blocked the driveway, and {name} didn't want to be rud
        "While studying, {name} accidentally discovered a portal to the year 192
        "The dog didn't eat the homework, but he did hide {name}'s shoes in the
    };

    int index = std::rand() % excuses.size();
    std::string selectedExcuse = excuses[index];

    // Placeholder replacement
    std::string target = "{name}";
    size_t pos = selectedExcuse.find(target);
    if (pos != std::string::npos) {
        selectedExcuse.replace(pos, target.length(), studentName);
    }

    std::cout << "\nGenerated Note:\n\"" << selectedExcuse << "\"\n" << std::endl
```

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

Pro-Tip: The "Buffer Flush" Issue

In more complex programs, if you use `std::cin >> someInt;` right before a `std::getline()`, the `getline` might appear to "skip" your input. This happens because `std::cin >>` leaves the newline character (`\n`) in the buffer.

To fix that specific scenario, you would use `std::cin.ignore()` before the `getline` to clear out that leftover character. In this current program, since we aren't mixing types, `getline` works perfectly on its own.

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

PROMPT AND OUTPUT 4:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

The program gives the same excuse every time I run it.
Fix this issue by:

Using srand(time(0)) properly
Ensuring random output each time...

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

The reason your program repeats the same excuse is that computers are **deterministic**. When you call `rand()`, it doesn't actually pick a number out of thin air; it follows a fixed mathematical formula starting from a "seed" value.

The Problem: A Static Seed

By default, the seed value is always **1**. Because the starting point is the same every time you launch the program, the "random" formula generates the exact same sequence of numbers. It's like playing a song from the beginning every time you turn on the radio—the first track will always be the same.

The Fix: `srand(time(0))`

To fix this, we use `std::srand()` (seed random) to change the starting point. By passing `time(0)` as the seed, we are using the **Unix Epoch time** (the number of seconds since January 1, 1970).

Since the time changes every single second, the "seed" is never the same twice. This forces the `rand()` function to start at a different point in its mathematical sequence, resulting in a different excuse each time you run the program.

Key Rules for `srand()`

- **Only call it once:** You should only call `srand` at the start of `main()`. If you put it inside a loop or call it multiple times per second, `time(0)` might return the same second, causing you to get the same "random" number over and over.
- **Include `<ctime>`:** Without this header, the compiler won't know what `time(0)` is.
- **Casting:** We use `static_cast<unsigned int>` because `time()` returns a large value (long long), and `srand` expects a smaller integer. This keeps the compiler from giving you a warning.

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

PROMPT AND OUTPUT 5:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Improve the program:

If the user enters an empty name, display a proper message
Avoid program crash or incorrect output
Keep the code simple and beginner-friendly.

Keep the code simple and beginner-friendly.

To make the program more professional and robust, we need to add a "guard" at the start. In programming, **validation** ensures that your code doesn't try to process "garbage" data (like an empty string), which can lead to silly-looking output or logic errors.

The Logic for Improvements

1. **The Validation Loop**: We use a `while` loop that keeps running as long as the `userName` is empty. This prevents the user from skipping the name prompt.
2. **The `.empty()` Method**: This is a built-in C++ string function. It's more readable than checking if the length is 0.
3. **Buffer Safety**: By using `getline` within this loop, we ensure that even if the user just hits "Enter," the program catches it, explains the issue, and asks again.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

C++

```
        if (studentName.empty()) {
            std::cout << "Error: Name cannot be blank. How can I write a note fo
        } else {
            // Success! We have a name, so we break out of the loop.
            break;
        }
    }

    // 3. STORAGE: 10 Excuse templates
    std::vector<std::string> excuses = {
        "Please excuse {name}, as their alarm clock was hijacked by a mischievou
        "Due to a sudden localized gravity failure, {name} was unable to leave t
        "The textbook required by {name} was unfortunately seized by pirates.",
        "I am writing to inform you that {name} is late because they had to solv
        "A rogue cloud followed {name} all morning, making travel impossible.",
        "The internet connection at {name}'s house was tired and needed a nap.",
        "Please forgive {name}; they accidentally entered a witness protection p
        "A group of ducks blocked the driveway, and {name} didn't want to be rud
        "While studying, {name} accidentally discovered a portal to the year 192
        "The dog didn't eat the homework, but he did hide {name}'s shoes in the
    };

    // 4. RANDOM SELECTION
    int index = std::rand() % excuses.size();
    std::string selectedExcuse = excuses[index];

    // 5. PLACEHOLDER REPLACEMENT
    std::string target = "{name}";
    size_t pos = selectedExcuse.find(target);

    if (pos != std::string::npos) {
        selectedExcuse.replace(pos, target.length(), studentName);
    }

    // 6. FINAL OUTPUT
    std::cout << "\n----------------------------------------------" << std::end
    std::cout << "OFFICIAL NOTE:" << std::endl;
    std::cout << "\"" << selectedExcuse << "\"" << std::endl;
    std::cout << "----------------------------------------------\n" << std::end

    return 0;
```

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

Why this is better for beginners:

- **User Experience (UX)**: Instead of the program just ending or showing a weird sentence like "Please excuse , as their alarm...", it gives a helpful error message.
- **Logical Flow**: The `while(true)` loop is a standard "best practice" for console apps. It creates a "trap" for bad data that the user can only escape by providing good data.
- **Code Resilience**: By checking `if (pos != std::string::npos)` before running the `replace` function, you protect the program from crashing if you (the programmer) ever forget to include the `{name}` tag in one of your templates.

Ask Gemini

Tools

Fast

PROMPT AND OUTPUT 6:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Add a feature:

After showing an excuse, ask the user if they want another one (y/n)
If yes, generate another excuse
If no, exit the program
Ensure proper input handling and no infinite loops.

To add a "Repeat" feature, we wrap the core logic of picking and printing an excuse inside a **do-while loop**. This structure is perfect for this task because it ensures the program runs at least once before asking the user if they want to continue.

The Logic

1. **Outer Loop (** `do-while` **):** This controls the entire flow. It keeps the program alive as long as the user's choice is 'y' or 'Y'.
2. **Inner Validation Loop (** `while` **):** We reuse the validation logic for the name to ensure the program doesn't proceed with blank data.
3. **Input Buffer Management**: After the user types 'y' or 'n', a newline character (`\n`) stays in the input buffer. We use `std::cin.ignore()` to clear it. Without this, the next `std::getline` for the name would see that "Enter" key and skip the prompt entirely!

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

C++

```
        } else {
            break;
        }
    }

    // 2. Storage: 10 Excuse templates
    std::vector<std::string> excuses = {
        "Please excuse {name}, as their alarm clock was hijacked by a mischie
        "Due to a sudden localized gravity failure, {name} was unable to lea
        "The textbook required by {name} was unfortunately seized by pirates.
        "I am writing to inform you that {name} is late because they had to
        "A rogue cloud followed {name} all morning, making travel impossible.
        "The internet connection at {name}'s house was tired and needed a nap
        "Please forgive {name}; they accidentally entered a witness protecti
        "A group of ducks blocked the driveway, and {name} didn't want to be
        "While studying, {name} accidentally discovered a portal to the year
        "The dog didn't eat the homework, but he did hide {name}'s shoes in
    };

    // 3. Random Selection
    int index = std::rand() % excuses.size();
    std::string selectedExcuse = excuses[index];

    // 4. Placeholder Replacement
    std::string target = "{name}";
    size_t pos = selectedExcuse.find(target);
    if (pos != std::string::npos) {
        selectedExcuse.replace(pos, target.length(), studentName);
    }

    // 5. Output
    std::cout << "\nOFFICIAL NOTE:\n\"" << selectedExcuse << "\"\n";

    // 6. Ask to repeat
    std::cout << "\nWould you like to generate another excuse? (y/n): ";
    std::cin >> choice;

    // FIX: Clear the newline character from the buffer so getline() doesn't
    std::cin.ignore(1000, '\n');

} while (choice == 'y' || choice == 'Y');
```

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

Key Improvements:

- `std::cin.ignore(1000, '\n')` : This is the most critical addition. It tells the computer to ignore everything in the input buffer (up to 1000 characters) until it hits a newline. This "resets" the stream for the next name entry.
- **Case Sensitivity**: By checking for both `'y'` and `'Y'`, the program is more user-friendly and doesn't quit just because Caps Lock was on.
- **The** `do-while` **Choice**: This structure is cleaner than an infinite `while(true)` loop because the exit condition (`choice == 'y'`) is clearly stated at the bottom of the code block.

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

PROMPT AND OUTPUT 7:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Explain how the student excuse generator program works step by step.
Also explain:

Why vector is used
How random selection works
How placeholder replacement is done

The student excuse generator is essentially a **template processing engine**. It takes raw data (the name), matches it with a pre-written scenario, and performs a surgical text replacement to create a custom output.

Here is a breakdown of the logic that powers this program.

---

1. Why a `vector` is used

In C++, a `std::vector` is a dynamic container that stores elements of the same type (in this case, `std::string`) in a linear sequence.

- **Indexing:** Every excuse in the vector is assigned a number (index), starting from **0**. This makes it incredibly easy to "point" to a specific excuse using a single integer.
- **Contiguous Memory:** Vectors store data in a single block of memory, which allows the computer to jump to the 10th excuse just as fast as the 1st one.
- **Dynamic Sizing:** Unlike a standard array, a vector can grow. If you wanted to add a feature where users could type in their own excuses during the program, a vector could expand to hold them on the fly.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

2. How Random Selection works

Computers are deterministic, meaning they normally do the exact same thing every time. To get "random" behavior, we use a two-step process:

- **Seeding (`srand`):** We use `std::srand(time(0))` to set a "seed." Think of this as choosing a specific starting point on a massive list of pre-calculated random numbers. By using the current time as the seed, the starting point changes every second.
- **The Modulo Operator (`%`):** The function `rand()` returns a very large number. To force that number into a range we can use, we divide it by the number of excuses we have and look at the **remainder**.

$$Index = RandomNumber \pmod{Number\ of\ Excuses}$$

If you divide any number by 10, the remainder can only be 0 through 9. This perfectly maps to our vector indices.

3. How Placeholder Replacement is done

This is the "Search and Destroy" part of the program. We don't just print the name followed by the excuse; we actually modify the string in the computer's memory.

- **Finding the Target:** The program uses `.find("{name}")`. The computer scans the string character by character. If it finds a match, it returns the **starting position** (the index) of that text.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

- **Replacing the Text:** The `.replace()` function takes three pieces of information:
  1. Where to start (the position found above).
  2. How many characters to remove (6, which is the length of `{name}`).
  3. What to put in that gap (the `studentName` variable).

Step-by-Step Execution Flow

1. **Initialize:** The program seeds the randomizer and enters the `do-while` loop.
2. **Input & Validation:** It uses `getline` to capture the name. If the name is empty, the `while` loop prevents the program from moving forward until the user provides one.
3. **Selection:** A random index is calculated, and a copy of that specific template is pulled from the `vector`.
4. **Transformation:** The program locates `{name}` inside the template and swaps it for the real name.
5. **Output:** The finished string is printed to the screen.
6. **Loop Control:** The program asks for a 'y/n' choice. It then uses `cin.ignore()` to "clean" the input buffer—this is crucial so the next name entry doesn't accidentally read an "Enter" key as a blank name.

If the user chooses 'y', the process resets; otherwise, the `do-while` condition fails and the

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.